System Testing and AI Workflow Verification Report

P27 — Development of an AI-Assisted Collaborative Worldbuilding Platform for Game Design
Group: COMP3888_M17_02_ZL · **Client:** Frank Lai
Execution: 26 October 2026 · Australia/Sydney
Status: Verification completed with open findings; three real Gemini smoke tests passed; broader AI quality evaluation pending.

1. Executive result

The executed tests provide evidence for the current platform's account, world-scoped authorization, proposal, version-history, relationship and AI-integration safeguards. They also identify an API validation defect and a login-page contrast finding. This is a team test report, not client acceptance or production-release approval.

Evidence layer	Actual outcome	Interpretation
Backend baseline leaf cases	68 passed; 1 failed; 3 skipped	Deterministic run; three live cases were later executed separately and passed
Node baseline runner totals	69 passed; 2 failed; 3 skipped; 74 reported tests	Includes two parent containers; one distinct defect: malformed registration email
Browser workflows and local read observation	13 passed; 0 failed; 0 skipped	12 browser scenarios plus one local API workload
Frontend unit tests	2 passed; 0 failed	Diff reconstruction and stale-suggestion checks
Static/build checks	3/3 passed	ESLint, TypeScript, production build
Backend coverage	96.45% lines; 89.14% branches	All 14 top-level backend runtime JS files included
Accessibility observation	1 rule violation; one affected element	Login page only; not a full WCAG audit
Separate real Gemini retest	3/3 passed; 0 skipped	Actual provider: gemini-3.8-flash; same run also passed two fixture tests

Open findings: D01 registration accepts an invalid email (Medium, proposed team severity); D02 a small login label has insufficient contrast (Low, proposed priority; axe classifies the rule impact as serious). Both remain unfixed in this tested application baseline. Mock AI responses are never presented as real Gemini results.

Updated AI status: A local key was subsequently configured and all three real-provider smoke scenarios passed. This report integrates that later successful run while retaining the deterministic baseline counts. The three historical skips are not current blockers, and the separately repeated fixture tests are not added as new unique cases.

2. Tested baseline and environment

- Git HEAD: `0f2d7cecb8d78da1d718259fc34312f9eec08a40`, with uncommitted test additions. The report concerns the working tree, not the clean commit alone. Non-secret file hashes and Git status are recorded in `evidence/run-metadata.json`.
- Runtime: v26.8.1, darwin/arm64; database reports version **26.7.0**, comment **Homebrew** (recorded verbatim).
- Frontend: React 19.2.6, vinext 1.0.0-beta.2 / Vite 8.0.13; backend: Express 5.2.1 and mysql2 3.24.2. Browser: Chromium 141.0.7390.37, Playwright 1.56.1.
- Coverage: c8 10.1.3, V8 instrumentation. Accessibility: axe 4.10.2.
- Browser run began 26/10/2026, 17:50:41; runner elapsed 32.25 seconds. Test-suite duration is not an API latency benchmark.
- Isolated generated databases were used for integration and browser writes. The application database was not written by this run. Final cleanup query found **0** matching temporary schemas.

3. Scope, strategy and test design

The September report focused on backend permissions, proposals, relationships and versions. The current scope retains that regression baseline and adds account/profile failure paths, sessions, search, browser recovery and the newer AI assistant/body-edit workflow. The earlier 81.02% line and 77.61% branch figures are historical context only: file scope, source and environment differ, so they are not a controlled before/after experiment.

Test type	Implementation	What it establishes
Unit / boundary	Node test runner; AI validation, graph paths, edit parsing, provider and diff checks	Deterministic rules and input partitions
API / integration	Actual Express HTTP handlers with isolated SQL schemas	Sessions, role decisions, state transitions, persistence and atomic version checks
Controlled failures	DB substitutes; provider substitutes; HTTP/network route interception	Safe error mappings, transaction lifecycle and UI retry without uncontrolled external failures
System / browser	Actual frontend and backend in Chromium	Representative visible user flows and recovery; AI requests clearly mocked where stated
Regression	Existing backend and frontend suites plus new cases	Current changes do not invalidate the exercised legacy workflows
Acceptance-style	Reconstructed user needs mapped to observable outcomes	Team verification; customer participation/sign-off not performed
Non-functional observation	Small local read workload, mobile layout, keyboard action and axe scan	Limited observations under stated local conditions

Design techniques and concrete inputs

Technique	Concrete example	Expected outcome
Equivalence partitioning	Guest, valid session, expired session; four roles and outsider	Reading/writing allowed only for the appropriate partition
Boundary-value analysis	Username 50/51; email 100/101; entity name 150/151	At-limit supported values accepted; over-limit rejected
Boundary-value analysis	Prompt 0/1/1999/2000/2001; references 4/5/6	Empty/oversized input rejected; supported maximum accepted
Boundary-value analysis	History 7/8/9; message text 3999/4000/4001; total 16000/16001	Per-message, count and total budgets enforced independently
Boundary-value analysis	AI edit input 5999/6000/6001; output 11999/12000/12001	Oversized input/replacements rejected without applying
State-transition testing	Draft → Pending → Approved/Rejected; Withdraw; Rejected → Draft	Legal transitions work; locked/repeated/illegal transitions fail
Permission decision table	Role × world visibility × membership × operation	Public visibility does not grant edit or history privileges
Concurrency/conflict	Competing approvals/restores; server advances while local editor stays open	One authorized current write succeeds; stale writes fail; input preserved
Failure injection	DB exception/duplicate, provider timeout/malformed output, request failure	Safe message, no unintended publication, explicit recovery

Boundary points are concentrated on important enforced limits; not every field/limit combination is exhaustive. SQL-like input checks establish behavior for those samples, not a complete injection audit. Detailed design and proposed live-AI scenarios are in `test-plan/TEST_PLAN.md`.

4. User-story traceability and acceptance scope

No approved, numbered User Story backlog was present in this checkout. **US-T IDs below are reconstructed themes**, based on current API/permission/AI documents and meeting minutes. They must not be described as official client story identifiers or a complete requirement denominator.

Theme	User need and acceptance criterion	Executed evidence	Outcome / boundary
US-T01	Register/sign in/update own account with valid input and protected session	Q01–Q08, Q13–Q14; UI01–02	Core paths pass; Q14 fails (D01)
US-T02	Create/browse worlds while preserving public/private access	Q09; legacy permissions; UI03	Exercised partitions pass
US-T03	Search and manage entity content with version protection	Q10–Q12; legacy permissions; UI07, UI12	Exercised search, write and recovery paths pass
US-T04	Store and navigate custom forward/reverse relationships	relationship-names; legacy permissions; UI03	Validation/storage/version checks and reverse display pass; full keyboard relationship editing not tested
US-T05	Privately draft, submit, withdraw and review contributions	legacy permissions; UI04	End-to-end Author submission/Owner approval and API transition checks pass
US-T06	Retain work during stale revisions, reload and access loss	Q11–Q12; legacy permissions; UI05–06, UI11	Local input preservation and explicit recovery pass
US-T07	Compare and restore historical published content	legacy permissions; UI07	History renders; restore API creates v3 from v1 and retains v1/v2. Compare/restore button interaction is not fully automated
US-T08	Ask AI with permission-scoped context and explicit failures	assistant/reference/provider/HTTP tests; B01–03; UI08, UI10; live retest	Deterministic tests and real conversation/consistency smoke pass; repeated quality evaluation pending
US-T09	Preview, dismiss or apply AI changes without auto-publication	assistant-edit/provider/HTTP tests; B04; frontend tests; UI09; live retest	Mocked UI lifecycle and actual-provider body-edit contract pass; broad rewrite quality unassessed
US-T10	Navigate usable forms and recover from failures	UI01–12; static/build checks; mobile/axe observation	Representative Chromium flows pass; contrast finding D02

Why this scope is useful and where sufficiency ends

The tests exercise the highest-impact platform risks: unauthorized content access, bypassed review, lost local text, duplicated or stale publication, inconsistent version/relationship state, and AI suggestions changing data without user control. Combining backend denial/state assertions with browser evidence is stronger than treating code coverage as a product-quality score.

The scope is sufficient to support the specific local functional claims in this report. It is **not sufficient to approve production deployment or assert that every user story is complete**. A verified backlog, real-provider evaluation, broader UI/accessibility checks and client acceptance remain necessary. No requirements-coverage percentage is calculated.

5. Backend and frontend execution results

Case group	Expected behavior	Actual result	Evidence
Q01–Q07, Q13	Registration limits/duplicates, login, profile ownership/current password, token rotation/expiry/logout	All pass; persisted profile/session state asserted	backend.junit.xml; account-quality.test.js
Q08	Reject malformed JSON, non-JSON, untrusted Origin and >1 MB body	400 / 415 / 403 / 413; existing session still usable	backend.junit.xml
Q09–Q12	Private defaults, visibility-aware search, stale writes and no partial entity creation	All pass; stale version and invalid relationship target preserve state	backend.junit.xml
Q14	Reject registration email `not-an-email`	**FAIL:** returns 201 and creates synthetic user instead of expected 400	D01; backend.tap / backend.junit.xml
F01–F04	DB/query/duplicate failures map to safe HTTP errors	Pass; 500 messages omit injected DB detail, duplicate maps to 409	failure-quality.test.js; backend.junit.xml
F05–F06	Failed/successful transaction releases connection and rolls back/commits	Pass; call sequence asserted using substitute connection	backend.junit.xml
B01–B04	AI prompt/reference/history/edit budgets accept/reject exact boundaries	All four grouped tests pass	boundary-quality.test.js
Existing permission integration	Authorization, drafts, transitions, Manager-review rules, versions, rollback, relationships and concurrency	Pass; **288 counted API/status assertions**, plus other content/state assertions	permissions.test.js; backend.tap
Existing AI suites	Context scoping, references/paths, draft permissions, provider errors, limits, edit contracts	Deterministic cases pass; baseline skipped three live cases, which passed in the separate retest	backend.junit.xml; live-ai-latest.junit.xml
Frontend unit tests	Reconstruct both diff versions; reject stale text/version/revision	2 pass	frontend-tests.log
Static/build checks	Lint/type checking/build exit successfully	All three exit 0	lint.log, typecheck.log, build.log

There are **72 backend leaf cases: 68 pass, 1 fail and 3 skip**. Node also counts the QAPI and QFAIL parent tests, reporting 74 tests: 69 pass, 2 fail and 3 skip. Q14 is the sole distinct failing leaf case. The 288 assertions are inside one broad legacy integration test, not 288 independently named cases or a coverage percentage.

The DB-failure tests substitute the database module while using actual HTTP handlers. They do not prove recovery from a real DB outage. The invalid-target integration test checks real database state after a rejected transaction. The pure relationship normalization suite was changed to avoid opening the configured application DB on import; product runtime code was unchanged.

6. Browser system results

ID	Scenario executed	Result
N01	observe 60 local reads at concurrency five without claiming a production SLA	PASSED
UI01	register mismatch, successful account creation and duplicate rejection	PASSED
UI02	login failure, keyboard password visibility and profile update	PASSED
UI03	guest/reader controls, private world exclusion and published reverse relationship	PASSED
UI04	author proposal submission and owner approval publish exact content	PASSED
UI05	stale proposal preserves local input, compares and explicitly recovers	PASSED
UI06	same-tab refresh restores unsaved proposal without publishing	PASSED
UI07	owner edits, compares history and restores as a newer version	PASSED
UI08	AI missing configuration gives disabled composer without changing published content	PASSED
UI09	mocked AI edit diff, dismiss, apply, stale protection and explicit save	PASSED
UI10	draft snapshot stale warning, refresh and mocked chat failure/retry	PASSED
UI11	revoked membership removes stale editor and retains copyable unsaved input	PASSED
UI12	search match/no result and small-screen accessibility observations	PASSED

The final run completed without unexpected or flaky outcomes; browser JSON contains 0 run-level errors. UI07 renders the history page and uses HTTP for restoration; it does not claim the comparison/restore controls were fully exercised. UI09–10 intercept the named AI endpoints with test doubles; all other relevant persistence and permission requests use the actual backend.

Test-harness corrections and retained evidence

Preliminary runs identified a missing hydration wait, an overly restrictive ready-element check, an exact-text locator that excluded a field label, and a DELETE helper missing JSON. Those harness errors were corrected and the entire 13-case browser set rerun. Earlier snapshots/results are retained in `evidence/initial-browser-harness`, `harness-second-run` and `browser-harness-run`; they are excluded from the final pass totals. They are not represented as fixed product defects.

The installed Playwright/Node combination emitted internal step-ID logging warnings for additional browser contexts. Assertions, screenshots and the JSON runner results completed successfully; detailed nested step traces may be incomplete. Chromium download extraction also required macOS ditto after the Node extractor stalled. These are tool-environment limitations, not application acceptance claims.

7. AI-specific verification and evidence boundaries

Area	Method	Observed / asserted behavior
Authentication/context access	Real backend integration and HTTP fixtures	Guests rejected; world/proposal guards reused; removed/private/cross-world references denied
Context assembly	Published fixture entities/graphs and SQL substitutes	Current versions, direct neighbors, selected references, intermediate paths, truncation and checked scope covered
Provider safeguards	Stubbed fetch/clock/abort	Credentials server-only; safe timeout/network/configuration/quota mappings; bounded output; no automatic retry; usage guard
Body-edit contract	Backend fixture tests and exact boundaries	Request owns original text/version; invalid, empty, oversized and token-truncated outputs rejected
In-flight changes	HTTP fixtures	Permissions rechecked after generation; stale baseline/revision blocks body edit
UI edit lifecycle	UI09, mocked edit endpoint + real entity persistence	Diff displays; dismiss preserves body; local edit makes suggestion stale; Apply modifies unsaved body only; Save increments persisted version
Snapshot/retry	UI10, mocked chat/config flag + real editor/context	Editor change warns; snapshot refresh clears stale warning; failed request retains input; explicit retry sends a second attempt
Missing configuration fallback	UI08, intentionally unconfigured isolated backend	Assistant reports unavailable and disables composer; this is a negative test, not the current local key status
Actual provider connection	Three opt-in tests with a subsequently configured local key	Real conversation, complete body rewrite and draft consistency all passed using gemini-3.8-flash

Successful real-provider retest

The three opt-in smoke tests were executed against the actual provider after configuring the local key. All passed; the same run's two deterministic HTTP fixture tests also passed (5/5 total). The configured model identifier was therefore validated through successful generation, not merely model discovery.

Live scenario	Actual outcome	Single test duration
General conversation	PASS	2.10 s
Complete body rewrite	PASS	2.70 s
Unsaved draft consistency	PASS	3.92 s

The ordinary reply correctly restricted entry to daylight. In the recorded consistency reply, the model identified midnight entry as a clear conflict with the night-entry ban, cited Harbour Regulations (ID 3, version 2), suggested daylight arrival/waiting offshore, and limited its conclusion to supplied context. The body-edit smoke verifies a usable, length-bounded replacement and server-owned original text; the full replacement was not logged for independent semantic review.

These are single synthetic trials, not an accuracy benchmark, billing/cost measurement or production latency SLA. General factual/creative quality, repeated contradiction accuracy and robustness to prompt injection remain unestablished. Prompt text tests alone cannot prove model compliance. The five proposed repeated rubric scenarios in TEST_PLAN.md remain **PLANNED / NOT RUN as a complete evaluation**; the contradiction smoke overlaps one theme but does not complete the five-scenario, three-trial plan. They are no longer blocked by a missing key. Evidence: live-ai-latest.tap, live-ai-latest.junit.xml and live-ai-success-summary.json. Earlier failed attempts remain historical evidence and are not the current connection status.

8. Measured backend code coverage

Metric	Covered / total (percentage)
Statements	1171 / 1214 (96.45%)
Lines	1171 / 1214 (96.45%)
Branches	731 / 820 (89.14%)
Functions	50 / 51 (98.03%)

Backend runtime file	Lines	Branches	Functions
ai-config.js	100%	100%	100%
assistant-api.js	100%	100%	100%
assistant-connections.js	100%	87.5%	100%
assistant-edit.js	100%	97.14%	100%
assistant-prompts.js	100%	100%	100%
assistant-router.js	93.75%	92.3%	50%
assistant.js	98.94%	90.66%	100%
content-api.js	97.65%	83.95%	100%
db.js	89.28%	50%	100%
gemini.js	100%	96.61%	100%
proposal-api.js	100%	85.42%	100%
relationships.js	94.91%	90.9%	100%
security.js	100%	91.3%	100%
server.js	89.71%	88.46%	100%

c8 `--all` includes all 14 top-level runtime JS files, including unexecuted eligible files. Tests, diagnostic scripts, dependencies, SQL and frontend source are excluded. The actual Express server is imported and exercised; coverage also includes paths exercised with substitutes. Frontend code coverage was **not measured**; passing browser or build checks do not supply it.

Statements and lines share a denominator in this V8-derived output. The function metric counts zero functions in server.js and some module files, producing 100% there; it must not be interpreted as every handler/feature being tested. Compact multi-statement lines also limit what line coverage establishes. Branch counts, assertions and scenario evidence are required for interpretation.

Remaining uncovered areas include DB connection failure logging, some member/list guards, profile exception/deletion paths, server standalone startup/error callbacks, relationship duplicate-write error mapping and AI preview/router paths. Open `coverage/index.html` for exact highlighted lines/branches. No coverage threshold was agreed with the client and no scores were inflated by excluding low-coverage runtime files.

Coverage was collected even though Q14 fails. A failing suite can execute many lines; high coverage does not cancel a failed acceptance criterion.

9. Findings, non-functional observations and retest status

D01 — Registration API accepts malformed email

- **Proposed severity:** Medium. **Status:** Open; not fixed in this baseline.
- **Precondition:** Isolated test schema, unauthenticated request.
- **Reproduction:** POST /register with username `Invalid Email`, email `not-an-email` and a nonempty synthetic password.
- **Expected:** HTTP 400; no account created, consistent with the browser email field and profile API validation.
- **Actual:** HTTP 201; response user email is `not-an-email`. Q14 records the failed assertion.
- **Impact:** Direct API clients can persist malformed account identifiers despite the UI constraint. This is an input-validation inconsistency, not demonstrated account takeover.
- **Suggested fix/retest:** Share server-side email validation across registration/profile; rerun Q14, valid/duplicate/boundary registration, login and UI01. No business-code fix was applied merely to produce a green report.

D02 — Login label contrast below the scanned threshold

- **Proposed priority:** Low; axe rule impact is **serious**. **Status:** Open automated finding, no independent human accessibility audit.
- **Scope:** Login page at 390 × 844, `.step-label` (“Welcome back”).
- **Actual:** axe reports contrast **4.45:1**, foreground #8b6d2f against #f8f5ec, 12px bold; rule expects 4.5:1 for this text.
- **Suggested fix/retest:** Adjust the label color and rerun the scan; inspect related labels and keyboard/zoom behavior. This one scan neither establishes nor disproves whole-application WCAG conformance.

Local read and layout observations

The actual backend handled **60** read requests at concurrency **5**, split equally among public world detail, entity detail and entity search, against a two-entity synthetic fixture. Results: **0 non-200 responses**, elapsed **417.4 ms**, p50 **5.20 ms**, p95 **334.44 ms**, max **353.65 ms**. End-to-end client timing includes request/body overhead; no dedicated endpoint warm-up, sustained load, large dataset, AI workload or acceptance threshold was used. Approximate local throughput (143.8 requests/s) is descriptive and must not be extrapolated to hosting capacity.

At 390px login width, document scroll width was 390px and viewport width 390px: the sampled page had no horizontal overflow. UI02 uses keyboard Enter to activate password visibility. This is a narrow interaction check, not an entire keyboard navigation or usability study.

10. Representative visual evidence

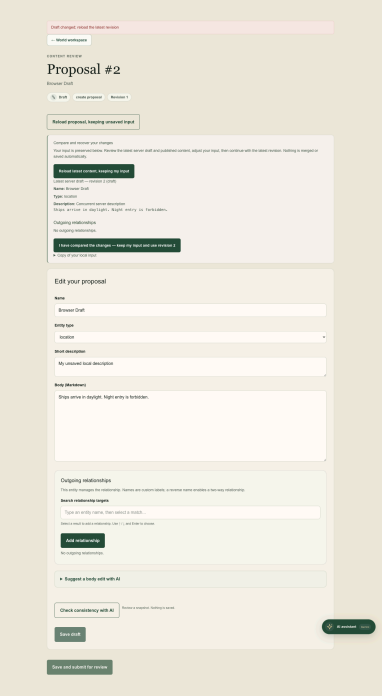


Figure 1. UI05: rejected stale save retains local description and displays the newer server revision for explicit recovery.

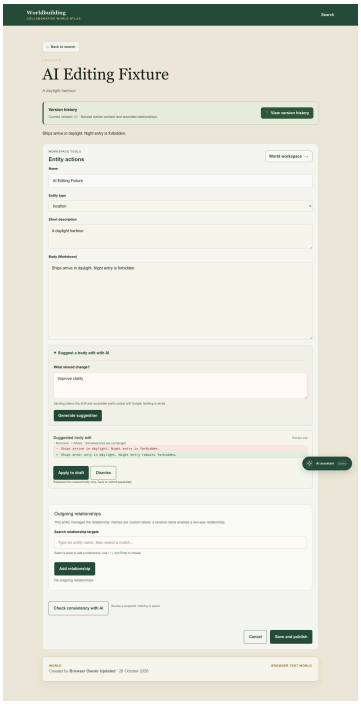


Figure 2. UI09: line-diff preview from a clearly mocked AI response. Persistence is separately asserted before Apply, after Apply and after Save.

All displayed accounts/worlds/content are synthetic. Individual scenario screenshots and machine-readable attachments are retained with the Playwright output.

11. Significance, limitations and reproduction

The current evidence extends the September backend baseline to account/profile failures and representative browser workflows, including the newer AI editor integration. It demonstrates exercised authorization, version/persistence and user-control safeguards and leaves two findings visible. It supports targeted local functionality claims; it does not certify a defect-free, secure, accessible or production-ready system.

Remaining priorities

1. Resolve and retest D01; review and correct D02.
2. Extend the three successful real-provider smoke tests to the documented five-scenario, repeated human-rubric evaluation. Retain synthetic requests/replies and keep keys out of evidence.
3. Confirm the official User Story backlog and map actual IDs/acceptance criteria; conduct and record client UAT.
4. Expand browser coverage to other engines/devices, complete keyboard/accessibility paths, relationship editing and UI version comparison/restore controls.
5. Test representative data volumes, agreed latency/load targets, deployment configuration, extended outage recovery and security controls before hosted acceptance.

Not included: exhaustive security/injection testing, full frontend source coverage, long-duration stress/availability, production hosting, real simultaneous browser co-editing or automatic merge, proof of AI semantic correctness, or a complete requirement-coverage percentage.

Evidence index

Artifact	Purpose
test-plan/TEST_PLAN.md	Acceptance themes, design techniques, passed live smoke and planned quality scenarios
reports/test-case-results.csv	72 backend baseline leaf outcomes, 13 browser/observation outcomes and 3 separate live retest outcomes
evidence/live-ai-latest.tap; live-ai-latest.junit.xml; live-ai-success-summary.json	Successful actual-provider retest; 3 live plus 2 repeated fixture tests
evidence/backend.tap; backend.junit.xml	Raw output, assertions, skip/failure and leaf-case timing
evidence/backend-coverage.log; backend-run.json	Instrumentation output, command, timestamps and exit code
coverage/index.html; coverage-summary.json; coverage-final.json; lcov.info	Inspectable and machine-readable backend coverage
evidence/browser-results.json; playwright-report/index.html	Final browser outcomes/attachments and interactive report
test-results/	Screenshots and any retained failure traces
evidence/frontend-checks.json and matching logs	Unit, lint, typecheck and build execution
evidence/accessibility-observations.json; local-read-observations.json	Raw rule findings and workload samples
evidence/database-cleanup-verification.json	Post-run isolated-database cleanup check
evidence/run-metadata.json; artifact-manifest.json	Source baseline and final evidence integrity

Reproduction commands

From the project root, with existing backend/frontend locked dependencies installed and a local MySQL account permitted to create/drop isolated test schemas:

```
cd testing
npm ci
npx playwright install chromium
node scripts/run-backend.cjs
node scripts/frontend-checks.cjs
npm run test:browser
```

```
node scripts/verify-cleanup.cjs
python3 scripts/export-cases.py
node scripts/metadata.cjs
node scripts/report.cjs
```

The backend runner is expected to exit nonzero until D01 is fixed. It still emits the evidence and coverage; do not discard its failure. Report rendering requires local pandoc and Playwright Chromium. The macOS executable fallback is in playwright.config.cjs; other systems use the normal Playwright installation. Required database/network/browser permissions depend on the local environment. Real-provider flags are forced off by the deterministic backend runner; live testing is a separate, explicitly configured activity.

Document basis

The supplied Worldbuilding_Test_Report_20260920.pdf (prepared 25 September from the 20 September run), the supplied assessment requirements screenshot, repository docs/EDITOR_API.md, docs/PERMISSIONS.md, docs/AI_ASSISTANT.md and Week 11 meeting minutes. These inform scope; numerical current results come from this run's stored evidence.